Q1 Report

Syed Ukkashah [23K-0055]

Parameters

My roll number is 23K-0055. Ignoring the letter and the hyphen, its digits are 2, 3, 0, 0, 5 and 5, which give the parameters below.

Symbol	Parameter	Formula	Working	Value
R	Last two digits of roll number		…55	55
D	Sum of digits mod 10	Σ digits mod 10	(2 + 3 + 0 + 0 + 5 + 5) = 15; 15 mod 10	5
Q	Normal daily queries	1,500 + 25 × R	1,500 + 25 × 55 = 1,500 + 1,375	2,875
N	LLM calls per query (first attempt)	5 + (R mod 4)	5 + (55 mod 4) = 5 + 3	8
p	Tool-call failure probability per call	0.04 + 0.01 × D	0.04 + 0.01 × 5	0.09
H	Heatwave days per month	4 + (R mod 4)	4 + (55 mod 4) = 4 + 3	7
B	Monthly inference budget (USD)	1,200 + 10 × R	1,200 + 10 × 55 = 1,200 + 550	$1,750
T	End-to-end latency target (s)	10 − (R mod 3)	10 − (55 mod 3) = 10 − 1	9 s

These parameters lead to the following values, which the rest of the report relies on.

Quantity	Value	Section
Expected input tokens per query, including retries	41,172.0	2.1
Expected output tokens per query, including retries	3,270.5	2.1
Expected LLM requests per query	8 + 0.09 × 7 = 8.63	2.1
Queries per month (30 days, H heatwave days at 5Q)	166,750	2.1
Input / output tokens per month	6.865 B / 0.545 B	2.1
Peak load, heatwave day, 12:00 to 14:00	41.93 queries/min = 362 RPM, 1.86 M TPM	2.3

My answer to the technical lead is to pay for an API. At SaafHawa's traffic the cheapest model that handles tool calls well is DeepSeek V4.1 Flash 1017, served through a hosted API for $308 a month against a budget of $1,750. Claude Haiku 5.5 is the fallback at $766 a month. Self-hosting the one open model with standard attention would take 3 GPUs and $2,354 a month, and it never becomes cheaper than this API anywhere between 10 and 200,000 queries a day. The GPUs have to be sized for a heatwave peak 10.9 times the average load, but they are billed around the clock. None of the candidates meets the 9 s latency target with an eight-turn loop either, so the agent loop needs redesigning as well (Section 2.6).

Section 1: Model comparison

I compared 7 closed-source models from 4 providers (OpenAI, Anthropic, Google, xAI) and 7 open-weight models from 6 providers (Mistral AI, Alibaba (Qwen), Google, OpenAI, DeepSeek, Z.ai). Each model runs in its non-reasoning or lowest reasoning setting, shown in brackets after its name. Every figure carries a source tag [S#], and I accessed all sources on 2026-11-09; the list follows the table. For the open-weight models I quote the cheapest public host for this workload's token mix, which is about 12.6 input tokens per output token, using OpenRouter's live endpoints API, and I name the host and its quantisation.

Table 1. Comparison of 14 models. Every figure is sourced, and every source was accessed on 2026-11-09.

Model (setting used)	Provider	Params (B) total / active	Context / max output (tokens)	$ per M tokens in / out / cached	Rate limit, new paid tier	TTFT / output tok/s	General benchmark(s)	Tool-use benchmark	Tools / JSON	Reasoning settings offered	Licence / commercial self-host
Closed-source models											
GPT-6 Luna (reasoning_effort=none)	OpenAI	not disclosed [S3]	1,050,000 / 128,000 [S3]	0.1 / 0.5 / 0.01 [S1] first-party API	5,000 RPM 2,000,000 TPM [S3]	1.15 s / 101 [S21;S22]	AA Intelligence Index v5.0.1 (non-reasoning): 9 [S21;S33] Terminal-Bench 4.0: 1.5 [S33]	AutomationBench-AA (non-reasoning): 9.1 [S33]	yes / yes [S3;S19]	none, low, medium, high, xhigh, max [S3;S19]	proprietary / no [S3;S19]
GPT-6.1 Terra (reasoning_effort=none)	OpenAI	not disclosed [S4]	1,050,000 / 128,000 [S4]	1.0 / 5.0 / 0.05 [S1] first-party API	5,000 RPM 1,000,000 TPM [S4]	1.63 s / 74 [S21;S22]	AA Intelligence Index v5.0.1 (non-reasoning): 31 [S21;S33] Terminal-Bench 4.0: 26.8 [S33]	AutomationBench-AA (non-reasoning): 51.0 [S33]	yes / yes [S4;S19]	none, low, medium, high, xhigh, max [S4;S19]	proprietary / no [S4;S19]
Claude Haiku 5.5 (effort=low (no non-thinking mode))	Anthropic	not disclosed [S6]	1,000,000 / 128,000 [S6]	0.1 / 0.5 / 0.01 [S5] first-party API	4,000 RPM 2,000,000 ITPM 400,000 OTPM [S8]	6.79 s / 232 [S21;S22]	AA Intelligence Index v5.0.1 (low): 19 [S21;S33] Terminal-Bench 4.0: 12.6 [S33]	AutomationBench-AA (low): 22.9 [S33]	yes / yes [S6;S9;S19]	adaptive thinking; effort low, medium (default), high, xhigh, max [S6;S9;S19]	proprietary / no [S6;S9;S19]
Claude Sonnet 5.5 (effort=low (no non-thinking mode))	Anthropic	not disclosed [S6]	1,000,000 / 128,000 [S6]	2.0 / 10.0 / 0.1 [S5] first-party API	4,000 RPM 2,000,000 ITPM 400,000 OTPM [S8]	2.4 s / 151 [S21;S22]	AA Intelligence Index v5.0.1 (low): 26 [S21;S33] Terminal-Bench 4.0: 20.7 [S33]	AutomationBench-AA (low): 49.4 [S33]	yes / yes [S6;S9;S19]	adaptive thinking; effort low, medium, high (default), xhigh, max [S6;S9;S19]	proprietary / no [S6;S9;S19]
Gemini 3.5 Flash-Lite (thinking_level=minimal (default))	Google	not disclosed [S18]	1,048,576 / 65,536 [S18]	0.3 / 2.5 / 0.03 [S10] first-party API	not published [S12]	4.08 s / 292 [S21;S22]	AA Intelligence Index v5.0.1: 12 [S21;S33] Terminal-Bench 4.0: 1 [S33]	AutomationBench-AA (default): 25.0 [S33]	yes / yes [S13;S19]	minimal (default), low, medium, high [S13;S19]	proprietary / no [S13;S19]
Gemini 3.8 Flash (thinking_level=low)	Google	not disclosed [S18]	1,048,576 / 65,536 [S18]	0.75 / 3.75 / 0.075 [S10] first-party API	not published [S12;S10]	3.74 s / 111 [S21;S22]	AA Intelligence Index v5.0.1 (low): 23 [S21;S33] Terminal-Bench 4.0: 10.1 [S33]	AutomationBench-AA (low): 36.5 [S33]	yes / yes [S13;S19]	low, medium (default), high [S13;S19]	proprietary / no [S13;S19]
Grok 4.20 0309 (non-reasoning) (non-reasoning model id)	xAI	not disclosed [S14]	1,000,000 / not stated [S14]	1.25 / 2.5 / 0.2 [S14] first-party API	1,776 RPM 10,000,000 TPM [S15]	not reported [S22]	IFBench (reasoning variant): 82.9 [S23]	AutomationBench-AA (not benchmarked by AA: deprecated model): not reported [S23]	yes / yes [S14;S19]	separate reasoning / non-reasoning model ids [S14;S19]	proprietary / no [S14;S19]

Model (setting used)	Provider	Params (B) total / active	Context / max output (tokens)	$ per M tokens in / out / cached	Rate limit, new paid tier	TTFT / output tok/s	General benchmark(s)	Tool-use benchmark	Tools / JSON	Reasoning settings offered	Licence / commercial self-host
Open-weight models											
Ministral 3 14B (instruct (non-reasoning))	Mistral AI	13.9 (dense) [S24;S19]	262,144 / 209,715 [S24;S19]	0.2 / 0.2 / 0.02 [S16;S19] host: Mistral	not published [S20]	2.54 s / 65 [S21;S22]	GPQA Diamond (vendor; reasoning-variant table): 71.2 [S24;S33] Terminal-Bench 4.0: 0 [S33]	AutomationBench-AA (default): 0.6 [S33]	yes / yes [S24;S19]	none in Instruct (separate Reasoning checkpoint) [S24;S19]	Apache 2.0 / yes [S24;S19]
Mistral Small 4 (reasoning off)	Mistral AI	119.4 total 6.5 active [S25;S19]	262,144 / 209,715 [S25;S19]	0.15 / 0.6 / 0.015 [S16;S19] host: Mistral	not published [S20]	1.77 s / 147 [S21;S22]	AA Intelligence Index v5.0.1 (reasoning): 4 [S22;S33] Terminal-Bench 4.0: 0 [S33]	AutomationBench-AA (reasoning; non-reasoning row not evaluated): 1.0 [S33]	yes / yes [S25;S19]	reasoning effort configurable per request [S25;S19]	Apache 2.0 / yes [S25;S19]
Qwen3.8 27B (thinking off)	Alibaba (Qwen)	27.8 (dense (hybrid linear/full attention)) [S26;S19]	262,144 / 65,536 [S26;S19]	0.051 / 0.362 / 0.006 [S19] host: PrimeIntellect (via OpenRouter), fp8	not published [S20]	5.31 s / 45 [S21;S22]	AA Intelligence Index v5.0.1 (non-reasoning): 10 [S21;S26;S33] Terminal-Bench 4.0: 0 [S33]	AutomationBench-AA (non-reasoning): 12.4 Terminal Bench 2.1 = 73.0 (vendor, S26) [S33]	yes / yes [S26;S19]	thinking on / off [S26;S19]	Apache 2.0 / yes [S26;S19]
Gemma 4 31B (thinking off)	Google	30.7 (dense (sliding + full attention)) [S27;S19]	262,144 / 235,929 [S27;S19]	0.1 / 0.34 / 0.1 [S19] host: CoreWeave (via OpenRouter), fp4	not published [S20]	3.9 s / 36 [S21;S22]	GPQA Diamond (vendor): 84.3 [S27;S33] Terminal-Bench 4.0: 0 [S33]	AutomationBench-AA (reasoning (AA default)): 4.9 Tau2 avg of 3 = 76.9% (vendor, S27) [S33]	yes / yes [S27;S19]	thinking on / off [S27;S19]	Apache 2.0 / yes [S27;S19]
gpt-oss-120b (reasoning=low (lowest))	OpenAI	116.8 total 5.1 active [S28;S19]	131,072 / 117,964 [S28;S19]	0.03 / 0.15 / 0.015 [S19] host: Near AI (via OpenRouter), fp4	not published [S20]	2.2 s / 144 [S21;S22]	Terminal-Bench 4.0: 0 [S33]	AutomationBench-AA (high; low row not evaluated): 0.2 [S33]	yes / yes [S28;S19]	low, medium, high [S28;S19]	Apache 2.0 / yes [S28;S19]
DeepSeek V4.1 Flash 1017 (non-reasoning)	DeepSeek	763.2 total not stated (MoE, 6 of 384 experts per token) active [S29;S31;S19]	1,048,576 / 131,072 [S29;S31;S19]	0.025 / 0.25 / 0.0015 [S19] host: Particle AI (via OpenRouter)	not published [S20]	1.32 s / 144 [S21;S22]	AA Intelligence Index v5.0.1 (non-reasoning): 19 [S21;S33] Terminal-Bench 4.0: 9.1 [S33]	AutomationBench-AA (non-reasoning): 48.8 [S33]	yes / yes [S29;S19]	non-reasoning, high, max [S29;S19]	MIT / yes [S29;S19]
GLM-5.3-Flash (default (reasoning; no non-reasoning variant benchmarked))	Z.ai	320 total 18 active [S30;S19]	1,048,576 / 131,072 [S30;S19]	0.034 / 0.083 / 0.03 [S19] host: Relace (via OpenRouter), fp4	not published [S20]	94.96 s / 49 [S21;S22]	AA Intelligence Index v5.0.1: 28 [S21;S33] Terminal-Bench 4.0: 32.8 [S33]	AutomationBench-AA (default): 60.4 [S33]	yes / yes [S30;S19]	reasoning (default) [S30;S19]	MIT / yes [S30;S19]

AutomationBench-AA is Artificial Analysis's agentic tool-use evaluation, scored in percent, and I use it as the tool-use benchmark. The AA Intelligence Index v5.0.1 combines ten evaluations. Where Artificial Analysis did not evaluate the setting I use, the table names the setting it did evaluate.

Table 2. Sources cited in Table 1, all accessed on 2026-11-09.

ID	Source	URL
S1	OpenAI API pricing (Standard tier)	https://developers.openai.com/api/docs/pricing
S2	OpenAI prompt caching guide	https://developers.openai.com/api/docs/guides/prompt-caching
S3	OpenAI model page: gpt-6-luna	https://developers.openai.com/api/docs/models/gpt-6-luna
S4	OpenAI model page: gpt-6.1-terra	https://developers.openai.com/api/docs/models/gpt-6.1-terra

ID	Source	URL
S5	Anthropic pricing	https://platform.claude.com/docs/en/about-claude/pricing
S6	Anthropic models overview	https://platform.claude.com/docs/en/about-claude/models/overview
S7	Anthropic prompt caching	https://platform.claude.com/docs/en/build-with-claude/prompt-caching
S8	Anthropic rate limits	https://platform.claude.com/docs/en/api/rate-limits
S9	Anthropic structured outputs	https://platform.claude.com/docs/en/build-with-claude/structured-outputs
S10	Google Gemini API pricing	https://aistudio.google.com/docs/pricing
S11	Google Gemini context caching	https://aistudio.google.com/docs/caching
S12	Google Gemini rate limits	https://aistudio.google.com/docs/rate-limits
S13	Google Gemini thinking	https://aistudio.google.com/docs/thinking
S14	xAI models and pricing	https://docs.x.ai/docs/models
S15	xAI consumption and rate limits	https://docs.x.ai/docs/key-information/consumption-and-rate-limits
S16	Mistral API pricing	https://docs.mistral.ai/inference/pricing
S17	Mistral prompt caching	https://docs.mistral.ai/inference/conversations/advanced/prompt-caching
S18	OpenRouter models API	https://openrouter.ai/api/v1/models
S19	OpenRouter per-model endpoints API	https://openrouter.ai/api/v1/models/{model-id}/endpoints
S20	OpenRouter API limits	https://openrouter.ai/docs/api-reference/limits
S21	Artificial Analysis LLM leaderboard	https://artificialanalysis.ai/leaderboards/models
S22	Artificial Analysis model pages	https://artificialanalysis.ai/models/{slug}
S23	Artificial Analysis IFBench	https://artificialanalysis.ai/evaluations/ifbench
S24	Hugging Face: mistralai/Ministral-3-14B-Instruct-2512	https://huggingface.co/mistralai/Ministral-3-14B-Instruct-2512
S25	Hugging Face: mistralai/Mistral-Small-4-119B-2603	https://huggingface.co/mistralai/Mistral-Small-4-119B-2603
S26	Hugging Face: Qwen/Qwen3.8-27B	https://huggingface.co/Qwen/Qwen3.8-27B
S27	Hugging Face: google/gemma-4-31B-it	https://huggingface.co/google/gemma-4-31B-it
S28	Hugging Face: openai/gpt-oss-120b	https://huggingface.co/openai/gpt-oss-120b
S29	Hugging Face: deepseek-ai/DeepSeek-V4.1-Flash-1017	https://huggingface.co/deepseek-ai/DeepSeek-V4.1-Flash-1017
S30	Hugging Face: zai-org/GLM-5.3-Flash	https://huggingface.co/zai-org/GLM-5.3-Flash
S31	Hugging Face model API (safetensors parameter count)	https://huggingface.co/api/models/{repo-id}
S32	RunPod GPU pricing (Pods; page updated 2026-11-04)	https://www.runpod.io/pricing
S33	Artificial Analysis leaderboard - Intelligence Index expanded into component evaluations (AutomationBench-AA; Terminal-Bench 4.0)	https://artificialanalysis.ai/leaderboards/models

Two values in the table could not be verified, and both affect the later analysis. The first is the number of reasoning tokens that Claude Haiku 5.5, Claude Sonnet 5.5, Gemini 3.5 Flash-Lite and Gemini 3.8 Flash produce at their lowest setting. These models cannot turn thinking off, and neither the providers nor Artificial Analysis publish how many

thinking tokens they average at low or minimal effort. I therefore add zero reasoning tokens for them, which makes their cost look lower than it is. If low effort produced even 300 thinking tokens per call, output per query would rise by about 80%, and the output part of the bill with it. The recommendation would stay the same, since Claude Haiku is already the more expensive of the two candidates and the Gemini and Sonnet models are over budget even without thinking tokens. Latency suffers less from this gap, because Artificial Analysis measures the time to the first answer token at that setting, and that time already includes the thinking.

The second is the rate limit a new paid account gets on Google Gemini (Tier 1) and on the open-weight hosts. Google shows per-model RPM and TPM only inside a signed-in AI Studio console. OpenRouter says paid models have no platform-level cap but that upstream providers apply their own limits, and those hosts do not publish them. I cannot confirm that the heatwave peak in Section 2.3 fits within these limits, so I mark them as unknown. The model I recommend is one of them, which makes a written rate-limit quota, or a load test at 362 RPM, a condition for launch.

A few smaller gaps appear as "not reported" in the table. Artificial Analysis stopped benchmarking Grok 4.20 after it was deprecated, so the table has no speed or tool-use score for it and only one benchmark. xAI's caching documentation page returns a 404 error, and DeepSeek does not state the active parameter count for V4.1 Flash.

Section 2: Calculations

The notebook Assignment_Beta_Question_no_1_23K-0055.ipynb holds every calculation, and I reproduce the working here. The notebook computes the latency budget (2.6) before 2.4 and 2.5 because both of those sections use its results.

2.1 Token volume

For each call k from 1 to N, with N = 8, the input is 2,000 + 800 × (k − 1) tokens and the output is 350 tokens, except for the final call, which produces 600. Calls 1 to N − 1 fail with probability p = 0.09, and a failed call is repeated once with the same input, so each of those calls runs 1 + p = 1.09 times on average. The final call is never retried.

Table 3. Tokens per LLM call, including the expected retries.

Call k	Input tokens	Output	Expected runs	Expected input	Expected output
1	2,000 + 800 × 0 = 2,000	350	1.09	2,180.0	381.5
2	2,000 + 800 × 1 = 2,800	350	1.09	3,052.0	381.5
3	2,000 + 800 × 2 = 3,600	350	1.09	3,924.0	381.5
4	2,000 + 800 × 3 = 4,400	350	1.09	4,796.0	381.5
5	2,000 + 800 × 4 = 5,200	350	1.09	5,668.0	381.5
6	2,000 + 800 × 5 = 6,000	350	1.09	6,540.0	381.5
7	2,000 + 800 × 6 = 6,800	350	1.09	7,412.0	381.5
8	2,000 + 800 × 7 = 7,600	600	1.00	7,600.0	600.0
Σ	**38,400**	**3,050**	**8.63**	**41,172.0**	**3,270.5**

```
Input per query  = first attempt + retries = 38,400 + 0.09 × 30,800 = 38,400 + 2,772.0 = 41,172.0
Output per query = 3,050 + 0.09 × 2,450 = 3,050 + 220.5 = 3,270.5
LLM requests per query = 8 + 0.09 × 7 = 8.63
```

```
Queries per month = (30 − H) × Q + H × 5Q = 23 × 2,875 + 7 × 5 × 2,875 = 66,125 + 100,625 = 166,750
Input tokens/month  = 166,750 × 41,172.0 = 6,865,431,000  (6.865 B)
Output tokens/month = 166,750 × 3,270.5 = 545,355,875  (0.545 B)
```

2.2 Monthly API cost, with and without prompt caching

Without caching, one query costs the expected input tokens times the input price plus the expected output tokens times the output price, divided by 10^6. The monthly cost is that figure times the number of queries in the month. With caching, the first 1,500 input tokens of every LLM request are the same system prompt and tool schemas, so 1,500 ×

8.63 = 12,945.0 input tokens per query are billed at the cached rate and the remaining 28,227.0 at the normal rate.

Writing to the cache costs extra. Traffic averages about two queries a minute even on normal days, so the shared prefix should stay warm, but to be safe I assume the prefix is rewritten once per cache lifetime for the whole month. That is 30 × 24 × 60 / TTL writes, each costing the write price minus the cached read price. Mistral does not state a TTL, so I assume 5 minutes.

I apply caching only where the provider documents a minimum cacheable prefix of 1,500 tokens or less: OpenAI (1,024), Anthropic (512) and Mistral (64). Gemini 3.x needs 4,096 tokens, xAI does not document its rules, and the third-party open-weight hosts publish no minimum, so I price those models without caching. Reasoning tokens count as zero at the chosen settings, for the reason given after Table 1.

Claude Haiku 5.5 shows the arithmetic. It charges $0.1 per million input tokens, $0.5 for output, $0.01 for cached input and $0.125 for a cache write, and its cache lasts 5 minutes.

```
No cache : (41,172.0 × 0.1 + 3,270.5 × 0.5) / 1e6 = $0.00575/query
           × 166,750 queries = $959.22/month
Cache    : (28,227.0 × 0.1 + 12,945.0 × 0.01 + 3,270.5 × 0.5) / 1e6 = $0.00459/query
           × 166,750 = $764.95
Writes   : 30×24×60 / 5 = 8,640 writes × 1,500 × (0.125 – 0.01) / 1e6 = $1.49
Total with caching = $766.44/month   (B = $1,750)
```

Table 4. Monthly API cost at Q = 2,875 a day and H = 7 (166,750 queries). Prices are in $ per million tokens, and the two latency candidates from 2.6 are highlighted. Cache rules come from [S2] OpenAI, [S7] Anthropic, [S11] Google, [S15] xAI, [S17] Mistral and [S19] the OpenRouter hosts.

Model	Side	$/M in / out / cached	Cache write $/M	Min. prefix	Cache TTL	Cache applies	No cache	With cache	≤ B? no / with
GLM-5.3-Flash	open	0.034 / 0.083 / 0.03	none	not published	not published	no	$279	$279	✓ / ✓
gpt-oss-120b	open	0.03 / 0.15 / 0.015	none	not published	not published	no	$288	$288	✓ / ✓
DeepSeek V4.1 Flash 1017	open	0.025 / 0.25 / 0.0015	none	not published	not published	no	$308	$308	✓ / ✓
Qwen3.8 27B	open	0.051 / 0.362 / 0.006	none	not published	not published	no	$548	$548	✓ / ✓
GPT-6 Luna	closed	0.1 / 0.5 / 0.01	0.125	1024	30 min	yes	$959	$765	✓ / ✓
Claude Haiku 5.5	closed	0.1 / 0.5 / 0.01	0.125	512	5 min (1 h opt.)	yes	$959	$766	✓ / ✓
Gemma 4 31B	open	0.1 / 0.34 / 0.1	none	not published	not published	no	$872	$872	✓ / ✓
Mistral Small 4	open	0.15 / 0.6 / 0.015	none	64	not stated	yes	$1,357	$1,066	✓ / ✓
Ministral 3 14B	open	0.2 / 0.2 / 0.02	none	64	not stated	yes	$1,482	$1,094	✓ / ✓

Model	Side	$/M in / out / cached	Cache write $/M	Min. prefix	Cache TTL	Cache applies	No cache	With cache	≤ B? no / with
Gemini 3.5 Flash-Lite	closed	0.3 / 2.5 / 0.03	storage $1.00/M tok/h	not listed; 3.x: 4,096	implicit; TTL not stated	no	$3,423	$3,423	✗ / ✗
Gemini 3.8 Flash	closed	0.75 / 3.75 / 0.075	storage $0.50/M tok/h	4096	implicit; TTL not stated	no	$7,194	$7,194	✗ / ✗
GPT-6.1 Terra	closed	1.0 / 5.0 / 0.05	1.25	1024	30 min	yes	$9,592	$7,544	✗ / ✗
Grok 4.20 0309 (non-reasoning)	closed	1.25 / 2.5 / 0.2	not documented	not documented (caching page 404)	not documented	no	$9,945	$9,945	✗ / ✗
Claude Sonnet 5.5	closed	2.0 / 10.0 / 0.1	2.5	512	5 min (1 h opt.)	yes	$19,184	$15,114	✗ / ✗

9 models fit the $1,750 budget without caching, and the same 9 fit with it: GPT-6 Luna, Claude Haiku 5.5, Ministral 3 14B, Mistral Small 4, Qwen3.8 27B, Gemma 4 31B, gpt-oss-120b, DeepSeek V4.1 Flash 1017 and GLM-5.3-Flash. Caching saves GPT-6 Luna 20%, GPT-6.1 Terra 21%, Claude Haiku 5.5 20%, Claude Sonnet 5.5 21%, Ministral 3 14B 26%, Mistral Small 4 21%, but it does not bring any model under the budget line. GPT-6.1 Terra, Claude Sonnet 5.5, Gemini 3.5 Flash-Lite, Gemini 3.8 Flash and Grok 4.20 0309 (non-reasoning) cost 2 to 9 times the budget even with caching.

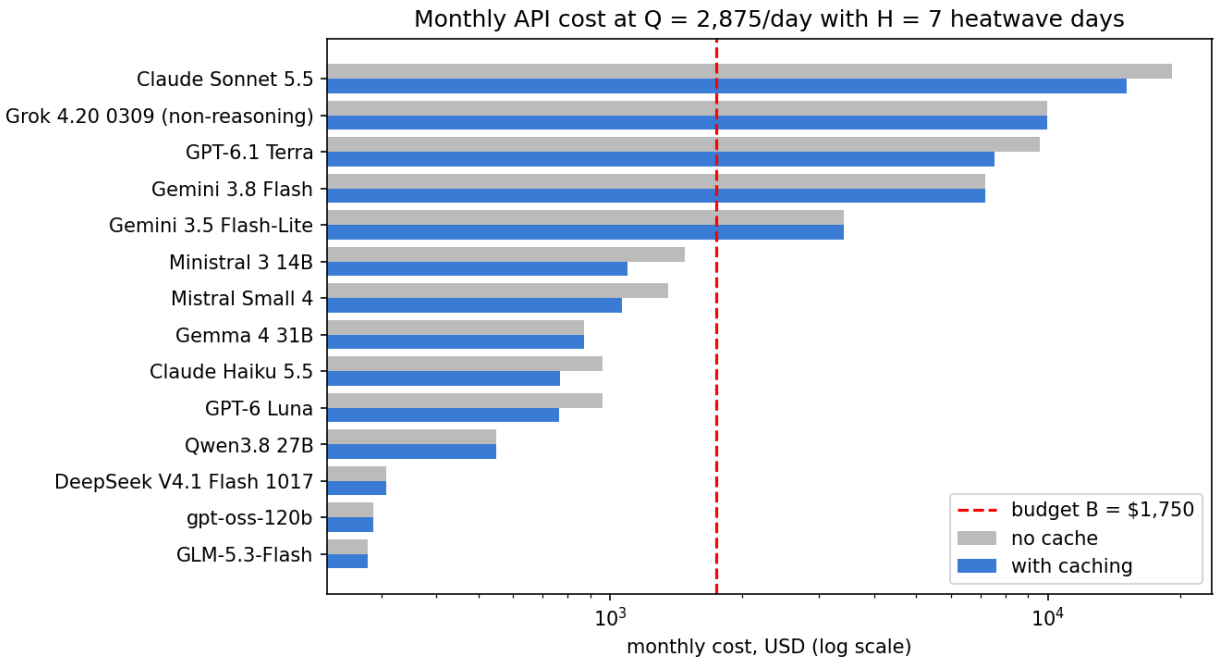


Figure 1. Monthly API cost with and without caching, on a log scale. The red line marks the budget B = $1,750.

2.3 Peak load and rate limits

On a heatwave day traffic rises to 5Q, and 35% of that day's queries arrive in the two hours between 12:00 and 14:00.

```
Heatwave day traffic     = 5 × Q = 5 × 2,875 = 14,375 queries
Queries 12:00 to 14:00   = 0.35 × 14,375 = 5,031.25 queries in 120 minutes
Peak queries per minute  = 5,031.25 / 120 = 41.93
Peak requests per minute = 41.93 × 8.63 LLM requests/query = 361.8 RPM
Peak input TPM           = 41.93 × 41,172.0 = 1,726,222
Peak output TPM          = 41.93 × 3,270.5 = 137,123
Peak total TPM           = 1,863,344
Uncached input TPM       = 41.93 × 28,227.0 = 1,183,476  (Anthropic counts only these toward ITPM)
```

Table 5. Peak load against each provider's limits for a new paid account (sources as in Table 1).

Model	Limit	Tier / note	Peak check
GPT-6 Luna	5,000 RPM; 2,000,000 TPM	Build tier (lowest listed); TPM is combined	fits
GPT-6.1 Terra	5,000 RPM; 1,000,000 TPM	Build tier (lowest listed); TPM is combined	EXCEEDS
Claude Haiku 5.5	4,000 RPM; 2,000,000 ITPM + 400,000 OTPM	Start tier; cache reads excluded from ITPM	fits
Claude Sonnet 5.5	4,000 RPM; 2,000,000 ITPM + 400,000 OTPM	Start tier; cache reads excluded from ITPM	fits
Gemini 3.5 Flash-Lite	not published	Tier 1 per-model limits only visible inside a signed-in AI Studio account	unknown (limits not published)
Gemini 3.8 Flash	not published	Tier 1 per-model limits only visible inside a signed-in AI Studio account; prices are promotional until 2026-12-31 ($1.50/$7.50 from 2027-01-01)	unknown (limits not published)
Grok 4.20 0309 (non-reasoning)	1,776 RPM; 10,000,000 TPM	Tier 0 (new account): 37 RPS and 10M TPM; RPM derived as 37 x 48 from the RPS = RPM/48 rule on the same page	fits
Ministral 3 14B	not published	no OpenRouter platform cap for paid models; Mistral tier page returned 404	unknown (limits not published)
Mistral Small 4	not published	no OpenRouter platform cap for paid models; Mistral tier page returned 404	unknown (limits not published)
Qwen3.8 27B	not published	no OpenRouter platform cap for paid models; upstream provider limits apply but are not published	unknown (limits not published)
Gemma 4 31B	not published	no OpenRouter platform cap for paid models; upstream provider limits apply but are not published	unknown (limits not published)
gpt-oss-120b	not published	no OpenRouter platform cap for paid models; upstream provider limits apply but are not published	unknown (limits not published)
DeepSeek V4.1 Flash 1017	not published	no OpenRouter platform cap for paid models; upstream provider limits apply but are not published	unknown (limits not published)
GLM-5.3-Flash	not published	no OpenRouter platform cap for paid models; upstream provider limits apply but are not published	unknown (limits not published)

GPT-6.1 Terra's new-account limit of 1,000,000 combined tokens per minute is below the 1.86 M TPM peak, so it would throttle SaafHawa at midday during a heatwave. GPT-6 Luna, both Claude models and Grok fit. Anthropic leaves cache reads out of its input limit, so for Claude the number to compare is 1,183,476 against 2,000,000. For Gemini and the open-weight hosts I cannot say either way, for the reason given after Table 1.

2.4 Self-hosting memory

For self-hosting I chose Ministral 3 14B Instruct, which is released under Apache 2.0. It is the only open-weight model in Table 1 with standard full grouped-query attention, with no sliding window and no linear or latent attention, so the assignment's KV-cache formula applies as written. Qwen3.8 mixes linear and full attention, Gemma 4 and gpt-oss combine sliding and global attention, and Mistral Small 4, DeepSeek and GLM use latent or sparse attention. The values below come from the text_config block of the model's config.json on Hugging Face [S24].

config.json field	Value
num_hidden_layers	40
num_key_value_heads	8
head_dim	128
num_attention_heads	32
sliding_window	None
parameters (safetensors total, incl. vision encoder) [S31]	13,945,031,680


```
"text_config": {
  "attention_dropout": 0.0,
  "head_dim": 128,
  "hidden_act": "silu",
  "hidden_size": 5120,
  "initializer_range": 0.02,
  "intermediate_size": 16384,
  "max_position_embeddings": 262144,
  "model_type": "ministral3",
  "num_attention_heads": 32,
  "num_hidden_layers": 40,
  "num_key_value_heads": 8,
  "rms_norm_eps": 1e-05,
  "rope_parameters": {
    "beta_fast": 32.0,
    "beta_slow": 1.0,
    "factor": 16.0,
    "llama_4_scaling_beta": 0.1,
    "mscale": 1.0,
    "mscale_all_dim": 1.0,
    "original_max_position_embeddings": 16384,
    "rope_theta": 1000000000.0,
    "rope_type": "yarn",
    "type": "yarn"
  },
```


Figure 2. The text_config block of config.json for mistralai/Ministral-3-14B-Instruct-2512 on Hugging Face, showing num_hidden_layers = 40, num_key_value_heads = 8 and head_dim = 128 (from https://huggingface.co/mistralai/Ministral-3-14B-Instruct-2512/blob/main/config.json).

Weight memory is the parameter count times the bytes per parameter.

```
FP16  : 13,945,031,680 × 2 bytes   = 27.89 GB
8-bit : 13,945,031,680 × 1 byte    = 13.95 GB
4-bit : 13,945,031,680 × 0.5 bytes = 6.97 GB
```

For the KV cache I use the longest context in the loop. The final call (N = 8) has 7,600 input tokens and generates 600 more, so the sequence reaches 8,200 tokens.

```
KV bytes = 2 × layers × kv_heads × head_dim × seq_len × bytes_per_value
         = 2 × 40 × 8 × 128 × 8,200 × 2 = 1,343,488,000 bytes = 1.343 GB
```

For the GPU I picked an NVIDIA L40S 48 GB (RunPod Secure Cloud) at $1.09 an hour on demand [S32]. At 4-bit the model needs only about 7 GB, so 48 GB leaves plenty of room. The card is also cheaper per concurrent sequence than an H100 80 GB, which costs $2.89 an hour and holds 48 sequences against the L40S's 26 at $1.09. I reserve 10% of the memory for activations and runtime overhead.

```
Usable for KV = 0.90 × 48 GB − 6.97 GB (4-bit weights) = 36.23 GB
Max sequences = floor(36.23 / 1.343) = 26
```

To count GPUs I need the number of sessions open at the same time. Little's law gives this as the arrival rate times the time each session holds its KV cache, and a session holds it for the whole query, tool waits included. I estimate that time with the 2.6 latency model, using Ministral 3 14B's measured speed (2.54 s TTFT, 65 tok/s [S21; S22]) as a stand-in for our own server. The result is 77.0 s per query.

```
Peak arrival rate   = 41.93 queries/min / 60 = 0.699 queries/s
Concurrent sessions = 0.699 × 77.0 s = 53.8
GPUs needed         = ceil(53.8 / 26) = 3
```

2.5 Break-even analysis

Self-hosting cost rises in steps because the number of GPUs is max(1, ceil(peak concurrency / 26)) and each GPU is billed for the full month, so the monthly cost is GPUs × $1.09 × 24 × 30 = GPUs × $784.8.

The assignment asks for a comparison with the 2.6 candidate that meets T after the architectural change, or with the faster one if neither does. Neither meets T, so the comparison is with DeepSeek V4.1 Flash 1017. Its

cost grows in a straight line with volume, because queries per month = $Q(30 - H) + 5QH = Q(30 + 4H) = 58Q$.

```
At our Q = 2,875: self-host = 3 × $784.8 = $2,354/month;  API = $308/month
Self-host $/query at large Q ≈ $1.09 × 720 / 26 × concurrency per daily query / 58 = $0.00974
API $/query (DeepSeek V4.1 Flash 1017) = $0.00185
Scan Q = 10 to 200,000/day: self-hosting is never cheaper, so there is no Q* against DeepSeek V4.1
Flash 1017
Average load = 166,750 / 43,200 min = 3.86 q/min; peak = 41.93; peak/average = 10.9
```

Table 6. Break-even volume Q* of self-hosting (step function, 26 sessions per GPU) against every API model.

API model	Side	API $/month at our Q	Q* (queries/day)	Q* / our Q
GLM-5.3-Flash	open	$279	never (up to 200k)	
gpt-oss-120b	open	$288	never (up to 200k)	
DeepSeek V4.1 Flash 1017	open	$308	never (up to 200k)	
Qwen3.8 27B	open	$548	never (up to 200k)	
GPT-6 Luna	closed	$765	never (up to 200k)	
Claude Haiku 5.5	closed	$766	never (up to 200k)	
Gemma 4 31B	open	$872	never (up to 200k)	
Mistral Small 4	open	$1,066	never (up to 200k)	
Ministral 3 14B	open	$1,094	never (up to 200k)	
Gemini 3.5 Flash-Lite	closed	$3,423	660	0.2
Gemini 3.8 Flash	closed	$7,194	320	0.1
GPT-6.1 Terra	closed	$7,544	300	0.1
Grok 4.20 0309 (non-reasoning)	closed	$9,945	230	0.1
Claude Sonnet 5.5	closed	$15,114	150	0.1

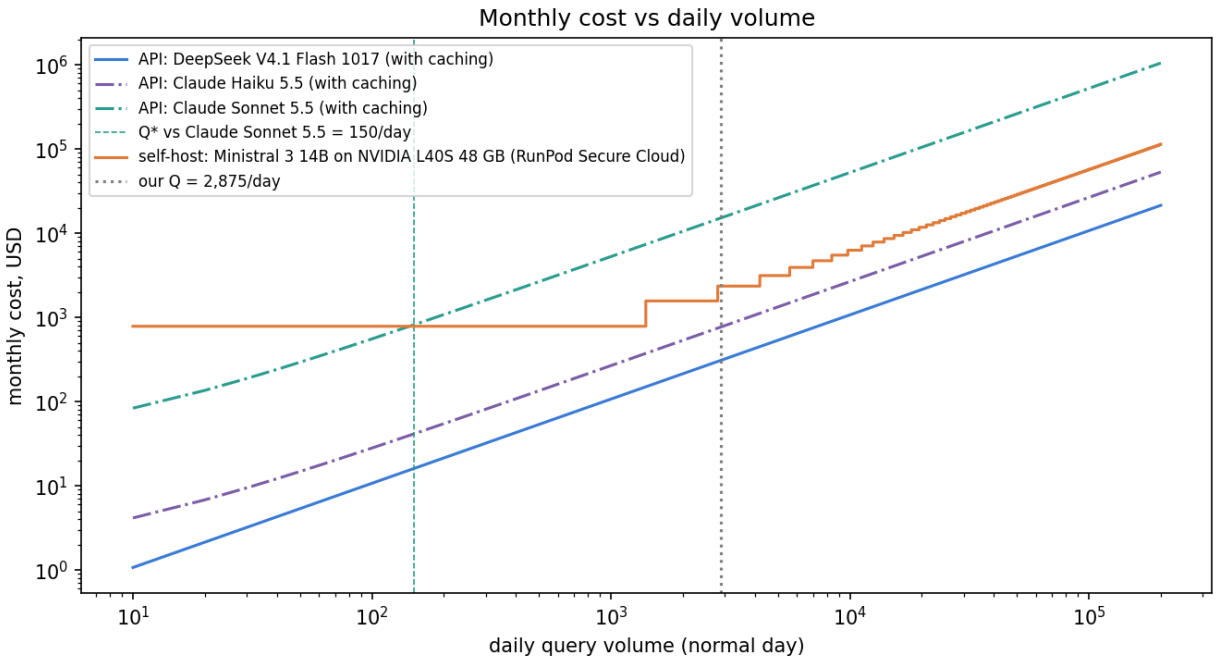


Figure 3. Monthly cost against daily query volume, log scales on both axes: the self-hosting step function against the API candidates.

Against DeepSeek V4.1 Flash 1017 at $308 a month, the API SaafHawa would actually use, self-hosting never breaks even. It costs about $0.00974 per query against the API's $0.00185, and because both costs grow in proportion to volume, more traffic never closes the gap. The GPUs have to be sized for the heatwave peak between 12:00 and 14:00, which is 10.9 times the average load, yet they are billed 24 hours a day for 30 days. Self-hosting beats only the premium closed models (Gemini 3.5 Flash-Lite, Gemini 3.8 Flash, GPT-6.1 Terra, Grok 4.20 0309 (non-reasoning) and Claude Sonnet 5.5), where Q* falls between 150 and 660 queries a day, far below our 2,875. Those models are over budget anyway and would not be chosen. GPUs that autoscale and are paid for only during peak hours, or a serving stack faster than the 65 tok/s per stream assumed here, could change the picture. For a WhatsApp assistant handling a few thousand queries a day, self-hosting is not a realistic option today.

2.6 Latency budget

To measure the tool delay I wrote latency_logger.py. Each run calls the air-quality API (OpenAQ v3, PM2.5 and PM10 for a Karachi station) and the weather API (Open-Meteo: temperature, humidity and wind) 20 times

each, and times the NOAA heat-index function and the knowledge-base lookup on my machine. Every timing goes into the raw log logs/tool_latency_log.csv, which is part of this submission.

Table 7. Tool latency measurements for successful calls (240 rows logged; 4 failed OpenAQ connections excluded).

Tool	n	Calls per day	Median (ms)	Mean (ms)	Max (ms)
air_quality	56	2026-11-09: 36, 2026-11-10: 20	372.014	525.107	919.492
heat_index	60	2026-11-09: 40, 2026-11-10: 20	0.011	0.013	0.178
kb_lookup	60	2026-11-09: 40, 2026-11-10: 20	0.037	0.305	15.679
weather	60	2026-11-09: 40, 2026-11-10: 20	529.303	531.492	718.605

```
Median of ALL 236 measurements (as instructed) = 0.262 ms; tool delay used = 0.000262 s
Median of the two network APIs only (sensitivity)    = 524.8 ms
```

Half of the measurements come from local functions that finish in well under a millisecond, so the pooled median is close to zero. A real agent waits about 0.5 to 0.7 s for each API tool, which means the pooled median understates the delay. I use it because the assignment requires it and report the API-only median (0.525 s) as a check. That check changes no conclusion, since LLM generation takes most of the time.

The calls run one after another, and each takes t_k = TTFT + extra prefill + output tokens ÷ output speed, with TTFT and output speed taken from Artificial Analysis at the chosen setting. Artificial Analysis measures TTFT on a fixed prompt, so I add the extra input at an assumed prefill rate of 5,000 tokens per second, giving extra prefill = (input_k − 2,000) / 5,000. This adds at most 1.1 s, at call 8. The agent catches an invalid tool call before the tool runs, so a retry repeats only the LLM call, and the expected retry delay is p × Σ t_k over k < N. The total is Σ t_k + (N − 1) × tool delay + retry delay.

I picked the two strongest candidates from the models that fit the budget with caching, have a single-call TTFT below T and have been benchmarked by Artificial Analysis. Among those I took the two with the highest AutomationBench-AA score, because choosing and calling tools is exactly what SaafHawa's loop does.

Table 8. Models eligible as latency candidates. The two highlighted rows have the highest tool-use scores.

Model	$/month (cache)	TTFT (s)	Output tok/s	AutomationBench-AA
DeepSeek V4.1 Flash 1017	$308	1.32	144	48.8
Claude Haiku 5.5	$766	6.79	232	22.9
Qwen3.8 27B	$548	5.31	45	12.4
GPT-6 Luna	$765	1.15	101	9.1
Gemma 4 31B	$872	3.9	36	4.9
Mistral Small 4	$1,066	1.77	147	1
Ministral 3 14B	$1,094	2.54	65	0.6
gpt-oss-120b	$288	2.2	144	0.2

Table 9. Per-call latency for DeepSeek V4.1 Flash 1017 (TTFT 1.32 s, 144 tok/s).

Call k	Input	TTFT (s)	Extra prefill (s)	Generation (s)	t_k (s)
1	2,000	1.32	0.00	350 / 144 = 2.43	3.75
2	2,800	1.32	0.16	350 / 144 = 2.43	3.91
3	3,600	1.32	0.32	350 / 144 = 2.43	4.07
4	4,400	1.32	0.48	350 / 144 = 2.43	4.23
5	5,200	1.32	0.64	350 / 144 = 2.43	4.39
6	6,000	1.32	0.80	350 / 144 = 2.43	4.55
7	6,800	1.32	0.96	350 / 144 = 2.43	4.71
8	7,600	1.32	1.12	600 / 144 = 4.17	6.61
Σ					**36.22**

```
Σ t_k = 36.22 s;  tools = 7 × 0.000262 = 0.0018 s;  retries = 0.09 × 29.61 = 2.67 s
Total = 36.22 + 0.0018 + 2.67 = 38.89 s   (T = 9 s)   [with API-only tool median: 42.56 s]
```

Table 10. Per-call latency for Claude Haiku 5.5 (TTFT 6.79 s, 232 tok/s).

Call k	Input	TTFT (s)	Extra prefill (s)	Generation (s)	t_k (s)
1	2,000	6.79	0.00	350 / 232 = 1.51	8.30
2	2,800	6.79	0.16	350 / 232 = 1.51	8.46
3	3,600	6.79	0.32	350 / 232 = 1.51	8.62
4	4,400	6.79	0.48	350 / 232 = 1.51	8.78
5	5,200	6.79	0.64	350 / 232 = 1.51	8.94
6	6,000	6.79	0.80	350 / 232 = 1.51	9.10
7	6,800	6.79	0.96	350 / 232 = 1.51	9.26
8	7,600	6.79	1.12	600 / 232 = 2.59	10.50
Σ					**71.95**

```
Σ t_k = 71.95 s;  tools = 7 × 0.000262 = 0.0018 s;  retries = 0.09 × 61.45 = 5.53 s
Total = 71.95 + 0.0018 + 5.53 = 77.48 s   (T = 9 s)   [with API-only tool median: 81.15 s]
```

Both candidates miss the 9 s target by a wide margin. Most of the time goes to generating output. Seven tool-calling turns of 350 tokens plus a 600-token answer come to about 3,050 tokens per query, and at 100 to 250 tok/s that alone takes 12 to 30 s before any TTFT is counted. Claude Haiku 5.5 also pays its 6.8 s TTFT at the low setting on every turn.

I tested the three suggested changes one at a time and then together. The first runs independent tool calls in parallel. Air quality, weather and the knowledge-base lookup do not depend on each other, so one LLM turn can request all three, which removes two LLM turns and two tool waits and cuts N from 8 to 6. The second streams the final response, so the user starts reading at the final call's first token and that call counts only its TTFT and prefill. The third routes the tool-selection turns 1 to N − 1 to the fastest model in the table that supports tool calling, which is DeepSeek V4.1 Flash 1017, while the final answer stays on the candidate. For DeepSeek V4.1 Flash 1017 itself routing changes nothing, since it is already that model.

Table 11. End-to-end latency in seconds after each architectural change.

Model	baseline	1. parallel tools, N=6	2. stream final answer	3. route turns 1..N-1	all three combined
DeepSeek V4.1 Flash 1017	38.89	28.47	34.72	38.89	24.31
Claude Haiku 5.5	77.48	57.15	74.89	42.78	29.78

Parallel tool calls give the largest single gain, taking DeepSeek V4.1 Flash 1017 from 38.89 s to 28.47 s, so that is the change I recommend. All three together bring it to 24.31 s, which is still above T. Faster generation would not get there on its own either: the combined design takes 23.78 s at 150 tok/s, 20.60 s at 200 tok/s, 17.42 s at 300 tok/s, 15.83 s at 400 tok/s, 14.24 s at 600 tok/s and 13.44 s at 800 tok/s. Reaching 9 s also needs shorter tool-selection turns. A compact JSON tool call is about 50 tokens, while this model assumes 350 for every turn.

Conclusions

SaafHawa will use 6.87 billion input and 0.55 billion output tokens a month. Of the 14 models, 9 fit the $1,750 budget, and prompt caching brings Claude Haiku 5.5 down from $959 to $766, a saving of 20%. GPT-

6.1 Terra, Claude Sonnet 5.5, Gemini 3.5 Flash-Lite, Gemini 3.8 Flash and Grok 4.20 0309 (non-reasoning) stay over budget even with caching.

The heatwave midday peak is 41.9 queries a minute, which works out to 362 requests and 1.86 million tokens a minute. GPT-6.1 Terra's 1 M TPM limit for a new account would throttle that. GPT-6 Luna, both Claude models and Grok fit, while Gemini and the open-weight hosts do not publish their limits.

Neither candidate meets the 9 s target. DeepSeek V4.1 Flash 1017 takes 38.9 s and Claude Haiku 5.5 takes 77.5 s, and nearly all of that is spent generating about 3,050 output tokens over 8 sequential turns. The measured tool delay hardly registers. Parallel tool calls help most (28.5 s), all three changes together reach 24.3 s, and the loop also needs short JSON tool turns.

Self-hosting Ministral 3 14B needs 6.97 GB for 4-bit weights and 1.34 GB of KV cache per 8,200-token session, so a 48 GB L40S holds 26 sessions. The peak needs about 54 concurrent sessions, which means 3 GPUs at $2,354 a month. That is over budget and 7.6 times the DeepSeek V4.1 Flash 1017 API bill, and it never breaks even. Ministral also scores just 0.6% on AutomationBench-AA, so it could not handle SaafHawa's tool calls in any case.

My recommendation is DeepSeek V4.1 Flash 1017 through an API, at $308 a month. It is the cheapest model with strong tool use, scoring 48.8% on AutomationBench-AA (close to Claude Sonnet 5.5 at low effort), and it has the fastest TTFT on the shortlist. The agent loop should be rebuilt around parallel tool calls, streamed answers and short JSON tool calls. The host does not publish its rate limits, which is the main risk, so a written quota has to be in place before launch. Claude Haiku 5.5 is the fallback. Its limits are documented and cover the peak, and it supports prompt caching, but it costs 2.5 times as much and is much slower at the low setting. The morning summary for municipal officers needs one more safeguard. A wrong figure there reaches people who make decisions for thousands of residents, so code should copy the numbers straight from the tool outputs instead of letting the model write them.

Along with this report I am submitting the executed notebook Assignment_Beta_Question_no_1_23K-0055.ipynb, the raw timing log logs/tool_latency_log.csv together with the latency_logger.py script that produced it, the Section 1 data in data/models.csv, data/sources.csv and data/snapshots/, the two figures in the figures folder, and my declaration of the AI tools I used.